BIG DATA COURSE

Data Warehouses & Cloud Analytics

From Dimensional Modeling to Snowflake — Theory & Practice

CHAPTER OVERVIEW

This chapter covers the foundational concepts of data warehousing, dimensional modeling, and the architecture of modern cloud data platforms. Students will learn both the theory and apply concepts hands-on using Snowflake.

Level: Undergraduate • Tool: Snowflake • Labs: Included

Learning Objectives

By the end of this chapter, students will be able to:

1. Define a data warehouse and explain the four core characteristics identified by Bill Inmon.
2. Distinguish between OLTP and OLAP systems and identify which is appropriate for a given scenario.
3. Describe the layers of a traditional data warehouse architecture.
4. Design a star schema by identifying fact and dimension tables from a business scenario.
5. Explain the difference between ETL and ELT and understand why ELT is preferred for cloud platforms.
6. Explain Snowflake's three-layer architecture and the concept of virtual warehouses.
7. Create Snowflake objects (warehouse, database, schema, table) and run SQL queries.
8. Load and query both structured and semi-structured (JSON) data in Snowflake.
9. Analyze query performance using the Query Profile and caching layers.

Practical Labs

This chapter includes four hands-on labs using Snowflake Snowsight. All labs reference the Snowflake Academia Technical Foundations Workbook (25K24-WIP). Download and unzip the lab
files before your lab session. Your instructor will provide access credentials.

Contents

1. Introduction to Data Warehousing

1.1 What is a Data Warehouse?

A data warehouse (DW or DWH) is a centralized repository designed to store, integrate, and analyze large volumes of data from multiple sources. Unlike the operational databases that power day-to-day business applications, a data warehouse is purpose-built for analytical querying — supporting reporting, business intelligence (BI), and data-driven decision-making.

Data warehouses aggregate data from diverse sources (sales systems, CRM platforms, flat files, APIs) and present it in a consistent, query-optimized format that analysts and business managers can easily explore.

1.1.1 Inmon's Four Defining Characteristics

Bill Inmon, widely regarded as the father of data warehousing, defined four essential properties that distinguish a data warehouse from other data stores:

Characteristic	Definition
Subject-Oriented	Organized around key business subjects — such as Sales, Customers, or Products — rather than around business applications or processes. This contrasts with operational databases, which are organized around transactions.
Integrated	Data arrives from many heterogeneous sources (ERP, CRM, flat files). A data warehouse harmonizes differences in naming conventions, units of measurement, data formats, and encoding so that data from all sources is consistent.

Time-Variant	Every record in a data warehouse contains a time element, allowing historical analysis. Data is never overwritten — new snapshots are appended. This supports trend analysis and period-over-period comparisons.
Non-Volatile	Once data is loaded into the warehouse, it is not modified or deleted under normal operations. Data is read-only for analytical purposes. This differs fundamentally from OLTP systems where records are frequently updated and deleted.

Key Concept

These four properties define why data warehouses exist: to provide a stable, consistent, historical view of business data that operational systems cannot easily provide.

1.1.2 A Brief History

The concept of separating operational and analytical data systems emerged in the 1980s. Barry Devlin and Paul Murphy at IBM introduced the concept of a 'business data warehouse' in 1988. Bill Inmon formalized the architecture in his 1992 book, Building the Data Warehouse. Ralph Kimball later proposed a more pragmatic, bottom-up approach centered on dimensional modeling — the foundation of most modern data warehouse designs.

The rise of cloud computing in the 2010s fundamentally changed data warehousing. Platforms like Snowflake (2012), Amazon Redshift (2012), and Google BigQuery (2010) moved warehousing to the cloud, offering elastic scalability, managed infrastructure, and consumption-based pricing.

1.2 Data Warehouse vs. Database vs. Data Lake

These three terms are often confused. Here is a concise comparison:

Dimension	Database (OLTP)	Data Warehouse (OLAP)	Data Lake
Purpose	Transactional operations	Analytics and reporting	Raw data storage
Data Type	Structured	Structured (integrated)	All types (raw)
Schema	Schema-on-write	Schema-on-write (optimized)	Schema-on-read
Latency	Real-time / milliseconds	Minutes to hours (batch)	Varies
Users	Applications, clerks	Analysts, BI tools	Data scientists
Storage Cost	High (optimized storage)	Medium	Low (object storage)
Query Style	Simple lookups, inserts	Complex aggregations	Exploratory
Example	MySQL, Oracle, PostgreSQL	Snowflake, Redshift, BigQuery	AWS S3, Azure Data Lake

Modern platforms like Snowflake blur these boundaries — Snowflake can function as a data warehouse, handle semi-structured data like a data lake, and expose data to real-time applications through Snowpipe and streams. This is why Snowflake describes its product as a 'Data Cloud' rather than simply a data warehouse.

2. OLTP vs. OLAP

2.1 Online Transaction Processing (OLTP)

OLTP systems are designed to manage and execute large numbers of short, atomic database transactions in real time. These are the operational backbone of any modern organization — they process orders, update account balances, record inventory changes, and handle user logins.

OLTP systems are optimized for write throughput: they minimize transaction latency, support high concurrency, and enforce ACID (Atomicity, Consistency, Isolation, Durability) properties to ensure data integrity. Their schema is highly normalized (often 3NF — Third Normal Form) to eliminate data redundancy and enable efficient updates.

OLTP Example

Consider an airline reservation system. When a customer books a flight, the OLTP system must:

1. Check seat availability (SELECT)
2. Reserve the seat (UPDATE)
3. Create a booking record (INSERT)
4. Process payment (external call + UPDATE)

2.2 Online Analytical Processing (OLAP)

OLAP systems are designed for complex analytical queries over large historical datasets. Rather than processing thousands of small write operations, OLAP workloads involve scanning millions or billions of rows to calculate aggregates, trends, and business metrics.

OLAP systems use different storage strategies than OLTP — often columnar storage, which reads only the columns needed by a query (rather than entire rows), dramatically improving performance for analytical aggregations. Schemas are intentionally denormalized to minimize joins and maximize query speed.

OLAP Example

The same airline's analytics team wants to answer: 'What was the average load factor per route, per quarter, over the past 5 years, and how does it compare to fuel costs?'
This query spans billions of booking records, joins multiple tables, and computes

2.3 Side-by-Side Comparison

Characteristic	OLTP	OLAP

Primary purpose	Process daily transactions	Support analytical decision-making
Query type	Simple CRUD (INSERT, UPDATE, DELETE, SELECT)	Complex aggregations (GROUP BY, JOIN, WINDOW)
Data volume per query	Rows (single record or small set)	Millions–billions of rows
Data model	Highly normalized (3NF)	Denormalized (star/snowflake schema)
Data currency	Current, real-time	Historical, time-stamped
Update frequency	Continuous (every second)	Batch loads (hourly, daily)
Response time target	< 1 second	Seconds to minutes acceptable
Storage model	Row-oriented	Column-oriented
Concurrency	Thousands of concurrent users	Tens to hundreds of analysts
Backup priority	High (transactional integrity)	Lower (can reload from source)

Common Exam Mistake

Students sometimes mix up OLTP and OLAP. Remember: OLTP = operations (transactions, real-time), OLAP = analytics (historical, aggregations). A company needs BOTH — the

3. Data Warehouse Architecture

3.1 Traditional Layered Architecture

A traditional enterprise data warehouse follows a layered architecture where data flows progressively from raw source systems to decision-ready information stores. Each layer serves a distinct purpose in cleansing, integrating, and optimizing data.

3.1.1 Layer 1 — Source Systems

Source systems are the operational databases, applications, and external feeds that generate the raw business data. These may include ERP systems (SAP, Oracle), CRM platforms (Salesforce), point-of-sale systems, web application logs, APIs, and flat files (CSV exports).
Each source typically uses its own schema, data types, and encoding conventions.

3.1.2 Layer 2 — Staging Area

The staging area is a temporary landing zone where raw data from source systems is copied without transformation. The primary purposes are to isolate source systems from the warehouse (minimizing impact on production systems), to allow auditing and debugging of raw data, and to serve as a starting point for transformations. Data in the staging area is typically purged after loading.

3.1.3 Layer 3 — Core Data Warehouse

The core (or enterprise) data warehouse integrates cleansed, transformed data from all sources into a unified, consistent data model. This layer enforces business rules, resolves key conflicts (e.g., different customer ID formats across systems), and maintains a complete historical record. This is where dimensional models (star schemas) typically reside.

3.1.4 Layer 4 — Data Marts

Data marts are subject-specific subsets of the core data warehouse, optimized for a particular business function or team. For example, a Sales data mart might contain pre-aggregated revenue metrics tailored for the Sales team's reporting tools, while an HR data mart contains headcount and payroll analytics. Data marts improve query performance and reduce complexity for end users.

3.1.5 Layer 5 — BI & Analytics

The final layer consists of the tools and interfaces that business users interact with: BI dashboards (Tableau, Power BI, Looker), self-service reporting tools, scheduled reports, and increasingly, machine learning models that consume warehouse data.

3.2 ETL vs. ELT

The ETL (Extract, Transform, Load) vs. ELT (Extract, Load, Transform) debate is central to

modern data warehousing. The choice of approach depends largely on the capabilities of the target data warehouse platform.

3.2.1 Traditional ETL

In the traditional ETL pattern, a dedicated ETL server or tool (such as Informatica, IBM DataStage, SSIS, or Talend) extracts data from source systems, performs all transformations in memory on the ETL server, and then loads the pre-processed data into the data warehouse.
This approach was the standard for on-premise warehouses.

```
-- Traditional ETL (conceptual pseudo-code)
-- Step 1: Extract (on ETL server)
raw_data = extract_from_source(connection=crm_db, query='SELECT * FROM
customers')

-- Step 2: Transform (in ETL server memory)
clean_data = clean_nulls(raw_data)
clean_data = standardize_country_codes(clean_data)
clean_data = apply_business_rules(clean_data)

-- Step 3: Load (into warehouse)
load_to_warehouse(target=dw.customers_dim, data=clean_data)
```

Limitation: The ETL server is a bottleneck. All data must pass through it, limiting throughput and scalability. As data volumes grow, the ETL server becomes expensive to scale and increasingly difficult to maintain.

3.2.2 Modern ELT

In the ELT pattern, raw data is loaded into the data warehouse first — as-is, with minimal transformation. The warehouse itself then performs the transformations using SQL, leveraging its massively parallel processing (MPP) engine. This approach has become the standard for cloud data warehouses like Snowflake.

```
-- Modern ELT in Snowflake (SQL-based transformation)
-- Step 1 & 2: Extract and Load raw data (e.g., COPY INTO from S3)
COPY INTO raw.customers
  FROM @my_s3_stage/customers/
  FILE_FORMAT = (TYPE = 'CSV' FIELD_OPTIONALLY_ENCLOSED_BY = '"');

-- Step 3: Transform inside Snowflake (using compute power)
CREATE OR REPLACE TABLE dwh.dim_customer AS
SELECT
    customer_id,
    UPPER(TRIM(first_name)) || ' ' || UPPER(TRIM(last_name)) AS full_name,
    COALESCE(country_code, 'UNKNOWN')                         AS country,
    CASE WHEN email LIKE '%@%' THEN email ELSE NULL END       AS valid_email,
    CURRENT_TIMESTAMP()                                       AS loaded_at
FROM raw.customers;
```

Why ELT Wins for Cloud DWs

Cloud data warehouses have virtually unlimited compute capacity on demand. Running transformations inside the warehouse (where the data already lives) is faster than moving data to an external ETL server, transforming it there, and then loading it back. Tools like dbt (data build tool) make SQL-based ELT transformations version-controlled, testable, and modular — similar to software

4. Dimensional Modeling

Dimensional modeling, pioneered by Ralph Kimball, is the dominant approach to designing data warehouse schemas. Its goals are simplicity, query performance, and intuitive design that business users can navigate. Dimensional models organize data around two types of tables: fact tables and dimension tables.

4.1 Fact Tables

Fact tables store the measurable, quantitative data about business events. Each row in a fact table represents a single business event or transaction — a sale, a flight, a web page view, a sensor reading. Fact tables are typically very large, containing millions to billions of rows.

4.1.1 Anatomy of a Fact Table

Component	Description
Foreign Keys (FKs)	References to surrounding dimension tables. These are the primary way of navigating from the fact to its context (e.g., which product, which customer, which date).
Measures (Facts)	The numeric values being measured: sales amount, quantity, revenue, cost, duration, count. These are the columns that get aggregated in queries.
Degenerate Dimensions	Dimensional attributes stored directly in the fact table because they have no additional attributes (e.g., transaction ID, invoice number).

4.1.2 Types of Facts

Type	Description
Additive	Can be summed across all dimensions. Example: Sales Amount — you can sum it across time, products, regions, etc.
Semi-Additive	Can be summed across some dimensions but not others. Example: Account Balance — summing across accounts makes sense, but summing across time periods does not.
Non-Additive	Cannot be meaningfully summed across any dimension. Example: Unit Price, Ratio, Percentage. These are typically stored for row-level calculations.

4.1.3 Sample Fact Table: Sales

```
-- FACT_SALES (one row per line item on a sales order)
CREATE TABLE fact_sales (
```

```
    date_key         INT NOT NULL,        -- FK to DIM_DATE
    product_key      INT NOT NULL,        -- FK to DIM_PRODUCT
    customer_key     INT NOT NULL,        -- FK to DIM_CUSTOMER
    store_key        INT NOT NULL,        -- FK to DIM_STORE
    order_number     VARCHAR(20),         -- Degenerate dimension
    quantity_sold    INT,                 -- Additive fact
    unit_price       DECIMAL(10,2),       -- Non-additive fact
    sales_amount     DECIMAL(12,2),       -- Additive fact (quantity * price)
    discount_amount DECIMAL(10,2)         -- Additive fact
);
```

4.2 Dimension Tables

Dimension tables provide the descriptive context that gives meaning to the facts. They answer the 'who, what, where, when, why, how' questions about a business event. Dimension tables are typically much smaller than fact tables (thousands to millions of rows) but tend to be wide — with many descriptive columns.

4.2.1 Sample Dimension: Product

```
-- DIM_PRODUCT
CREATE TABLE dim_product (
    product_key      INT PRIMARY KEY,    -- Surrogate key (warehouse-assigned)
    product_id       VARCHAR(20),        -- Natural/business key from source
    product_name     VARCHAR(100),
    category         VARCHAR(50),
    subcategory      VARCHAR(50),
    brand            VARCHAR(50),
    unit_cost        DECIMAL(10,2),
    launch_date      DATE,
    is_active        BOOLEAN
);
```

Surrogate Keys

Dimension tables use surrogate keys (warehouse-generated integers) rather than the natural keys from source systems. This protects the warehouse from changes in source systems (e.g., if the CRM renumbers customer IDs), supports slowly changing dimensions, and allows multiple source systems to be integrated without key conflicts.

4.2.2 Slowly Changing Dimensions (SCDs)

Dimension data changes over time — a customer moves to a new city, a product changes category, an employee changes department. How should the data warehouse handle these changes? Ralph Kimball defined three main SCD strategies:

SCD Type	Strategy and Use Case
Type 1 — Overwrite	The old value is simply overwritten with the new value. No history is retained. Use when history is unimportant (e.g., correcting a typo in a product name). Simple but loses historical context.
Type 2 — Add New Row	A new row is added with the new attribute values. The old row is flagged as inactive with an end date. This preserves complete history. Use when historical accuracy matters (e.g., tracking customer address changes for sales attribution).
Type 3 — Add New Column	A new column is added to store the previous value alongside the current one (e.g., previous_city, current_city). Provides limited history (only 'before and after'). Rarely used in practice.

```
-- SCD Type 2: Customer dimension with history tracking
CREATE TABLE dim_customer (
    customer_key     INT PRIMARY KEY,   -- Surrogate key
    customer_id      INT,               -- Natural key from CRM
    customer_name    VARCHAR(100),
    city             VARCHAR(50),
    country          VARCHAR(50),
    segment          VARCHAR(30),
    effective_date   DATE,              -- When this version became active
    expiry_date      DATE,              -- When this version was superseded
(NULL if current)
    is_current       BOOLEAN            -- TRUE for the current version
);
```

4.3 Star Schema

The star schema is the most widely used dimensional model layout. It places a single fact table at the center, surrounded by directly connected dimension tables. The resulting diagram resembles a star — hence the name.

Key characteristics of a star schema:

- Dimension tables are denormalized — all descriptive attributes for a subject are in a single wide table, even if this creates some redundancy.
- Queries are simple: typically a single join from the fact table to each needed dimension.
- Optimized for read performance — fewer joins means faster queries.
- Intuitive for business users and BI tools, which can auto-generate queries.

Star Schema vs. Normalized Schema

A key trade-off: star schemas are faster to query but use more storage (redundancy). A normalized relational schema is storage-efficient but requires

For analytical workloads scanning millions of rows, query speed matters far more than storage savings — which is why star schemas dominate in data

4.4 Snowflake Schema (Dimensional Concept)

Note: In this section, 'snowflake schema' refers to a dimensional modeling concept — not to be confused with Snowflake, the data platform covered in later sections.

A snowflake schema extends the star schema by normalizing dimension tables. Instead of a single flat dimension table, hierarchical relationships within dimensions are broken into separate related tables. For example, a Product dimension might be split into Product, Subcategory, and Category tables.

Aspect	Comparison
Storage	Snowflake schema uses less storage (normalization eliminates redundancy); star schema uses more.
Query Performance	Star schema is generally faster (fewer joins); snowflake schema requires more joins for the same query.
Complexity	Snowflake schema is more complex to design and query; star schema is simpler.
BI Tool Compatibility	Star schema works more naturally with most BI tools (drag-and-drop query builders).
Recommendation	In practice, star schemas are preferred for most data warehouse implementations. Snowflake schemas are used when storage is a critical constraint or when dimension tables are extremely large.

5. Snowflake: A Cloud-Native Data Warehouse

5.1 Overview

Snowflake is a cloud-native data warehousing platform, founded in 2012 and available on Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP). Unlike traditional data warehouses that bundle storage and compute together, Snowflake's architecture fully separates these concerns — a design decision that enables elastic scalability, multi-workload concurrency, and a consumption-based pricing model.

Snowflake describes itself as a 'Data Cloud' — a platform that goes beyond traditional data warehousing to support data lakes, data engineering, data applications, and AI/ML workloads within a single, unified platform.

5.2 Three-Layer Architecture

Snowflake's architecture consists of three distinct, independently scalable layers:

5.2.1 Layer 1 — Database Storage

All data stored in Snowflake resides in cloud object storage (AWS S3, Azure Blob Storage, or GCS depending on which cloud the Snowflake account runs on). Snowflake automatically manages this storage — users never interact with the underlying files directly.

Key storage characteristics:

- Data is stored in a compressed, columnar format using Snowflake's proprietary Hybrid Columnar Storage (HCS).
- Data is automatically divided into immutable micro-partitions (compressed chunks of 50-500 MB of uncompressed data), enabling efficient partition pruning during queries.
- Metadata about micro-partitions (min/max values per column, row counts, etc.) is stored in the Cloud Services layer and used to optimize query execution without reading the actual data.
- Storage is billed separately from compute — data stored does not consume compute credits.

5.2.2 Layer 2 — Query Processing (Virtual Warehouses)

Compute in Snowflake is provided by Virtual Warehouses (VWs) — named clusters of compute resources (CPU, memory, and local SSD disk). Key characteristics:

- Each virtual warehouse operates independently — multiple VWs can simultaneously query the same data without interference.

- VWs come in T-shirt sizes: XS, S, M, L, XL, 2XL, 3XL, 4XL. Each size roughly doubles the compute resources (and credit consumption rate) of the previous size.
- VWs auto-suspend after a configurable period of inactivity (e.g., 60 seconds) and auto-resume when a new query is submitted, eliminating waste when compute is idle.
- VWs can be scaled out horizontally using multi-cluster configurations to handle high concurrency.

```
-- Create a virtual warehouse
CREATE WAREHOUSE analytics_wh
  WITH WAREHOUSE_SIZE = 'X-SMALL'
  AUTO_SUSPEND = 60          -- Suspend after 60 seconds of inactivity
  AUTO_RESUME  = TRUE        -- Automatically resume on query submission
  INITIALLY_SUSPENDED = TRUE -- Start suspended (saves credits)
  COMMENT = 'Warehouse for analyst team queries';

-- Resize on the fly (no downtime, no data movement)
ALTER WAREHOUSE analytics_wh SET WAREHOUSE_SIZE = 'SMALL';

-- Manually suspend / resume
ALTER WAREHOUSE analytics_wh SUSPEND;
ALTER WAREHOUSE analytics_wh RESUME;
```

5.2.3 Layer 3 — Cloud Services

The Cloud Services layer is the intelligence layer of Snowflake — a collection of services that coordinate all activities across Snowflake. These services run on Snowflake-managed compute infrastructure and are included in the standard service cost (no separate configuration needed).

Cloud Services includes:

- Authentication and access control (RBAC — Role-Based Access Control)
- Infrastructure management and coordination
- Query parsing, optimization, and compilation
- Transaction management and ACID compliance
- Metadata management (schema information, statistics, micro-partition metadata)
- Security: encryption at rest and in transit, column-level security, row access policies

Key Insight: Separation of Storage and Compute

Traditional data warehouses (Oracle Exadata, Teradata, Netezza) bundle storage and compute in physical hardware. To scale, you buy more hardware — an expensive, slow process.

Snowflake separates these: storage scales automatically and indefinitely (it is just cloud object storage). Compute scales instantly, independently, and you only pay

5.3 Snowflake Object Hierarchy

Snowflake organizes all objects in a clear hierarchy, with role-based access control applicable at each level:

Object Level	Description

Organization	The top-level container for all Snowflake accounts within a company. Enables cross-account data sharing and usage management.
Account	An isolated Snowflake instance with its own storage, compute, users, and security policies. Most companies operate with one account per environment (dev/test/prod).
Database	A logical grouping of schemas. Roughly equivalent to a database in traditional RDBMS. Multiple databases can exist in one account.
Schema	A namespace within a database, grouping related tables, views, and other objects. Common pattern: one schema per data domain or processing stage (e.g., RAW, STAGING, ANALYTICS).
Table / View	Standard relational tables (permanent, transient, temporary) and views. Snowflake also supports dynamic tables and materialized views.
Stage	A named reference to a file location — either internal (Snowflake-managed storage) or external (S3, Azure Blob, GCS). Used for loading and unloading data.
Virtual Warehouse	Compute cluster. Defined at the account level; referenced by sessions when executing queries or DML.

5.4 Snowflake Query Performance

One of Snowflake's key differentiators for analytical workloads is its multi-tiered caching system and query optimization capabilities. Understanding these helps explain why the same SQL can have dramatically different performance characteristics.

5.4.1 Three-Tier Caching

Cache Type	How It Works
Metadata Cache	Query metadata (table row counts, min/max column values, micro-partition counts) is stored in the Cloud Services layer. Queries that only need metadata (e.g., SELECT COUNT(*)) complete in milliseconds without spinning up a virtual warehouse — no compute credits consumed.
Result Cache	Results of completed queries are cached in Cloud Services for 24 hours. If the exact same query is resubmitted and the underlying data has not changed, Snowflake returns the cached result instantly — no compute credits consumed. The

	cache is shared across users in the same account.

Data Cache (Local Disk Cache)	When a virtual warehouse reads micro-partitions from remote storage, it caches them on its local SSD disk. Subsequent queries on the same VW that access the same micro-partitions read from this fast local cache rather than remote storage, significantly reducing I/O latency.

5.4.2 Partition Pruning

Because Snowflake stores metadata about the min/max values for every column in every micro-partition, it can skip entire partitions that cannot possibly satisfy a query's WHERE clause predicates. This is called partition pruning and is one of the most important performance mechanisms.

```
-- Example: Query with a date predicate
-- If DIM_DATE.year has values 2020-2024 per micro-partition,
-- Snowflake only scans partitions where year could be 2023

SELECT
    product_name,
    SUM(sales_amount) AS total_sales
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_product p ON f.product_key = p.product_key
WHERE d.year = 2023          -- Enables partition pruning on fact_sales
  AND p.category = 'Electronics'
GROUP BY product_name
ORDER BY total_sales DESC;
```

6. Hands-On Labs — Snowflake in Practice

Before You Start

Download and unzip the lab files provided with the Snowflake Academia Technical Foundations Workbook (25K24-WIP). You will find a USERS folder containing sub-folders with usernames — locate your username and note the SQL files inside.
Your instructor will provide your Snowflake account URL and temporary credentials.

Lab 1 — Introduction to Snowflake and Snowsight

Scenario

You are working for Snowbear Air, a fictional airline. Your task is to set up a development environment in Snowflake, create core database objects, and run your first queries against airline data.

What You Will Do

1. Log in to Snowsight and configure Multi-Factor Authentication (MFA) using the Passkey method.
2. Explore the Snowsight UI: the navigation bar, Workspaces, Projects, and the user menu.
3. Upload your Lab 1 SQL file to a Workspace and set your session context (role, database, schema, warehouse).
4. Create a virtual warehouse, database, schema, and table using SQL.
5. Insert sample data and run SELECT queries.
6. Set user defaults (default role and warehouse) in User Settings.

Key SQL from Lab 1

```
-- Create a virtual warehouse
CREATE OR REPLACE WAREHOUSE my_wh
  WAREHOUSE_SIZE = 'X-SMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE;

-- Create a database and schema
CREATE OR REPLACE DATABASE snowbear_air_db;
CREATE OR REPLACE SCHEMA snowbear_air_db.dev;

-- Set context for subsequent queries
USE WAREHOUSE my_wh;
USE DATABASE snowbear_air_db;
USE SCHEMA dev;

-- Create a sample table and query it
```

```
SELECT * FROM flights LIMIT 10;
```

Lab 2 — Loading Structured Data

What You Will Learn

Snowflake's primary mechanism for bulk data loading is the COPY INTO command. Data files are first uploaded to a stage (an internal or external file location), then loaded into a Snowflake table.

Key Concepts

Concept	Explanation
Stage	A named location for data files. Internal stages are Snowflake-managed; external stages reference S3, Azure Blob, or GCS buckets.
File Format	A reusable object defining how files should be parsed (CSV delimiter, JSON encoding, Parquet schema, etc.).
COPY INTO	The SQL command that loads data from a stage into a Snowflake table. Supports CSV, JSON, Parquet, Avro, ORC, and XML.
ON_ERROR	Controls behavior when load errors occur: CONTINUE (skip bad records), SKIP_FILE (skip the whole file), ABORT_STATEMENT (stop and roll back).
GZIP Support	Snowflake automatically decompresses GZIP (.gz) files during COPY INTO — no pre-processing required.

```
-- Create an external stage pointing to an S3 bucket
CREATE OR REPLACE STAGE my_s3_stage
  URL = 's3://my-bucket/data/'
  CREDENTIALS = (AWS_KEY_ID = '...' AWS_SECRET_KEY = '...');

-- Define a CSV file format
CREATE OR REPLACE FILE FORMAT csv_format
  TYPE = 'CSV'
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  SKIP_HEADER = 1
  NULL_IF = ('NULL', 'null', '');

-- Load data from stage into table
COPY INTO fact_sales
FROM @my_s3_stage/sales/2024/
  FILE_FORMAT = (FORMAT_NAME = 'csv_format')
  ON_ERROR = 'SKIP_FILE';
```

```
-- Validate before loading (check for errors first)
COPY INTO fact_sales
FROM @my_s3_stage/sales/
  FILE_FORMAT = (FORMAT_NAME = 'csv_format')
  VALIDATION_MODE = RETURN_ERRORS;
```

Lab 3 — Caching and Query Performance

What You Will Learn

Lab 3 focuses on understanding why queries run at different speeds and how to diagnose and improve query performance using Snowflake's built-in tools.

The Query Profile

Snowflake's Query Profile is a visual execution graph (DAG — Directed Acyclic Graph) that shows how a query was executed. Each node in the graph represents a processing step (table scan, join, aggregation). For each node, you can see: time spent, bytes scanned, rows produced, and whether the result came from cache.

In Lab 3, you will access the Query Profile by running a query, then clicking the query ID in the History panel. You will observe the differences between:

- A first-run query (reads from remote storage — higher latency)
- A re-run of the same query (uses Result Cache — near-instant)
- A query with a selective WHERE clause (partition pruning reduces scan)
- A query without a WHERE clause (full table scan)

```
-- Run this query twice. Observe the difference in execution time.
-- Second run uses the Result Cache.
SELECT
    d.year,
    d.month,
    SUM(f.sales_amount) AS monthly_revenue
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
GROUP BY d.year, d.month
ORDER BY d.year, d.month;

-- Compare with EXPLAIN to see the execution plan before running
EXPLAIN
SELECT * FROM fact_sales WHERE date_key BETWEEN 20230101 AND 20231231;
```

Lab 4 — Semi-Structured Data and Cortex AI

Semi-Structured Data in Snowflake

Snowflake natively stores and queries semi-structured data formats (JSON, Avro, ORC, Parquet, XML) using a special data type called VARIANT. A VARIANT column can hold any JSON-like structure, including nested objects and arrays.

```
-- Create a table with a VARIANT column
CREATE TABLE user_events (
    event_id   NUMBER AUTOINCREMENT PRIMARY KEY,
    event_time TIMESTAMP,
    payload    VARIANT   -- Stores raw JSON
);

-- Query using dot notation (simple field access)
SELECT
    payload:user_id::INT          AS user_id,
    payload:event_type::STRING    AS event_type,
    payload:device:type::STRING   AS device_type   -- Nested field
FROM user_events;

-- Flatten an array field into separate rows
SELECT
    payload:order_id::STRING  AS  order_id,
    f.value:item_sku::STRING  AS  item_sku,
    f.value:quantity::INT     AS quantity
FROM orders,
LATERAL FLATTEN(input => payload:line_items) f;
```

Snowflake Cortex — AI Inside the Warehouse

Snowflake Cortex provides built-in LLM (Large Language Model) functions that run directly inside Snowflake — no external API calls, no data leaving your account. These functions allow you to apply AI capabilities to text data using standard SQL.

Cortex Function	What It Does
SENTIMENT(text)	Analyzes text and returns a score from -1.0 (very negative) to 1.0 (very positive). Useful for customer review analysis, support ticket triage, social media monitoring.
TRANSLATE(text, from_lang, to_lang)	Translates text between languages using an embedded multilingual LLM. Supports dozens of language pairs.
SUMMARIZE(text)	Produces a concise summary of a long text document (article, report, support ticket).
EXTRACT_ANSWER(question, context)	Given a question and a text passage, returns the most likely answer extracted from the passage (extractive QA).

```
-- Sentiment analysis on customer reviews
```

```
SELECT
    review_id,
    review_text,
    SNOWFLAKE.CORTEX.SENTIMENT(review_text) AS sentiment_score,
    CASE
        WHEN SNOWFLAKE.CORTEX.SENTIMENT(review_text) > 0.3 THEN 'Positive'
        WHEN SNOWFLAKE.CORTEX.SENTIMENT(review_text) < -0.3 THEN 'Negative'
        ELSE 'Neutral'
    END AS sentiment_label
FROM customer_reviews
LIMIT 100;
```

7. Review Questions

These questions are designed to consolidate your understanding of the chapter material. Attempt each question before checking your notes.

Conceptual Questions

7. List and briefly explain Inmon's four defining characteristics of a data warehouse. For each characteristic, give a one-sentence example of how it manifests in a real-world airline data warehouse.

8. A retail company operates a website that processes 50,000 orders per hour. Their analytics team needs to generate weekly sales reports, demand forecasting models, and customer segmentation analyses. Identify which system (OLTP or OLAP) is appropriate for each of these requirements, and explain your reasoning.

9. In the layered data warehouse architecture, what is the specific purpose of the staging area? Why is it important not to load data directly from source systems into the core data warehouse?

10. Explain the difference between ETL and ELT. In your answer, explain why ELT has become the preferred approach for cloud data warehouse platforms like Snowflake.

11. Design a star schema for a hospital analytics data warehouse. Identify: (a) at least one meaningful fact table and its measures, (b) at least four appropriate dimension tables with example attributes, and (c) two foreign key relationships between the fact and dimension tables.

Snowflake Architecture Questions

12. Explain Snowflake's three-layer architecture. What are the benefits of separating the storage layer from the compute layer? Give one concrete example of a scenario where this separation provides a clear advantage over traditional bundled architectures.

13. A data analytics team of 50 analysts uses Snowflake. The team runs peak workloads during business hours (9 AM – 5 PM) and the warehouse is idle overnight and on weekends. Explain how virtual warehouse AUTO_SUSPEND

and AUTO_RESUME features impact the company's Snowflake costs, and suggest an appropriate AUTO_SUSPEND setting with justification.

14. Describe Snowflake's three-tier caching system (Metadata Cache, Result Cache, Data Cache). For each cache type: explain when it is used, whether it consumes compute credits, and give an example query that would benefit from it.

Lab Application Questions

15. You are loading a 10 GB CSV file from an S3 bucket into Snowflake using COPY INTO. The file contains some rows with data quality issues. Write the SQL command you would run first to identify error rows before committing the load, and explain what VALIDATION_MODE = RETURN_ERRORS does.

16. You have a VARIANT column named event_data containing JSON records. One such record looks like: {"user": {"id": 1001, "name": "Alice"}, "actions": [{"type": "click", "page":
"home"}, {"type": "purchase", "amount": 59.99}]}. Write SQL to extract: (a) the user name, (b) the purchase amount from the actions array, and (c) use FLATTEN to return one row per action.

17. In Lab 3, you observed that re-running the same query returns results much faster the second time. Explain which caching mechanism is responsible, and describe under what conditions this cache would be invalidated (causing Snowflake to re-execute the query).

8. Chapter Summary

This chapter covered the foundational theory and practical application of data warehousing, from classical concepts to modern cloud platforms. Here are the key takeaways:

Topic	Key Takeaway
Data Warehouses	Purpose-built analytical systems defined by four properties: subject-oriented, integrated, time-variant, and non-volatile. They are separate from, and fed by, OLTP systems.
OLTP vs OLAP	OLTP handles real-time transactions (optimized for writes). OLAP handles analytical queries over historical data (optimized for reads). Both are necessary in a modern data architecture.
DW Architecture	Data flows through layers: Source → Staging → Core DW → Data Marts → BI. Modern cloud DWs often collapse these layers through elastic compute.
Dimensional Modeling	Star and snowflake schemas organize data into fact tables (measures) and dimension tables (context). Surrogate keys and SCD strategies handle change over time.
ETL vs ELT	ETL transforms outside the warehouse (bottleneck for large data). ELT loads first, then transforms inside the warehouse using its full compute power — the standard for cloud DWs.
Snowflake Architecture	Three decoupled layers: Storage (S3/Blob/GCS), Compute (Virtual Warehouses), Cloud Services (orchestration, security, optimization). Storage and compute scale independently.
Query Performance	Snowflake uses three cache tiers (metadata, result, data) and partition pruning to minimize compute usage and maximize query speed.
Semi-Structured Data	Snowflake's VARIANT type and dot notation querying allow JSON, Parquet, and similar formats to be stored and queried natively alongside structured tables.
Cortex AI	Built-in LLM functions (SENTIMENT, TRANSLATE, SUMMARIZE, EXTRACT_ANSWER) run SQL-accessible AI directly inside Snowflake — no external APIs needed.

Further Reading

- Ralph Kimball & Margy Ross: The Data Warehouse Toolkit, 3rd Edition
- Bill Inmon: Building the Data Warehouse, 4th Edition
- Snowflake Documentation: docs.snowflake.com

- dbt Documentation (ELT tooling): docs.getdbt.com
- Snowflake University: learn.snowflake.com